Arquitetura de computadores

Prova 1 de Arquitetura de computadores

Feito por: Felipe Echeverria Vilhalva

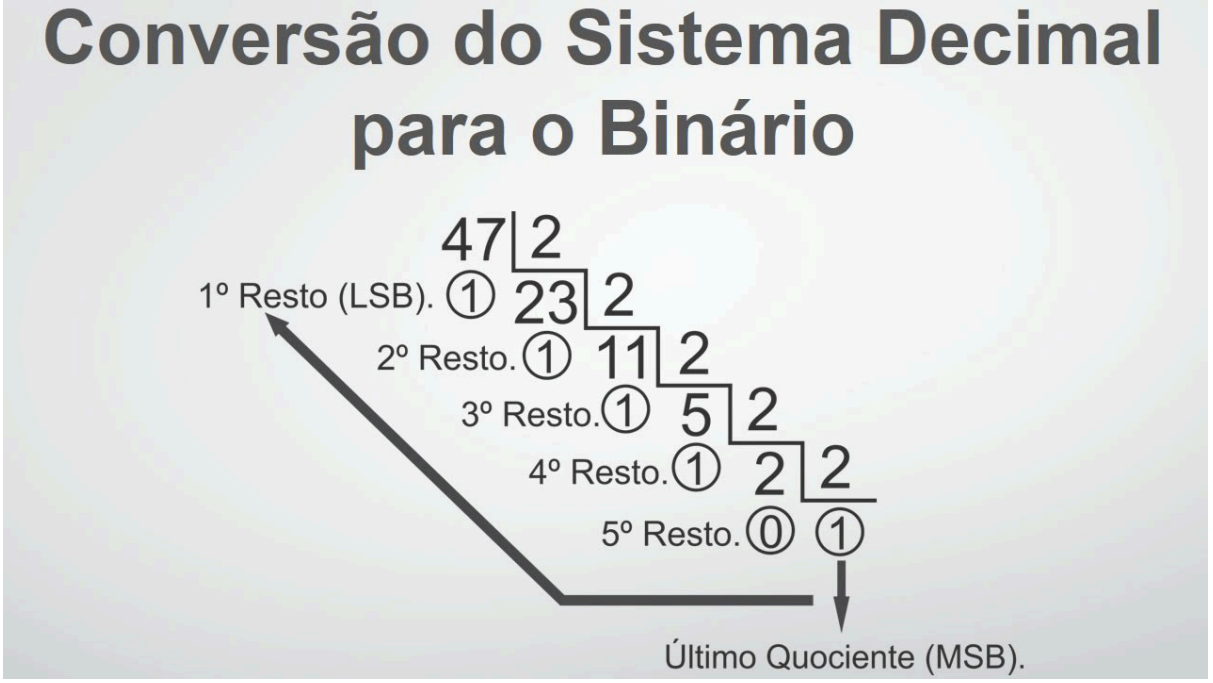
Um breve resumo

1. Conversão para Binário/Hexadecimal

A conversão entre binário e hexadecimal é direta porque o hexadecimal usa base 16, que é uma potência de 2. Cada dígito hexadecimal corresponde a 4 bits binários. Isso facilita a conversão ao simplesmente substituir grupos de quatro dígitos binários por um dígito hexadecimal.

2.1. Conversão decimal para binário

Para converter um número decimal em binário, é necessário dividir o quociente por 2 até não ser mais possível, depois só pegar o resto da direita para esquerda e agrupar.



2.2 Conversão binário para decimal

Para converter um número binário para decimal, pegamos o n número de bits do binário, e k para o bit, e somamos com , assim:

para um número binário:

$$k_i k_{i+1} k_{i+2} \dots k_n$$

temos,

$$\sum_{i=0,j=n}^{n,0} k_j \cdot 2^i$$

exemplo: 1101(lembrando de começar da direita para esquerda)

$$1 \cdot 2^4 + 0 \cdot 2^3 + 1 \cdot 2^2 + 1 \cdot 2^1 + 0 \cdot 2^0 = 16 + 0 + 4 + 2 + 0 = 22$$

2.3 Conversão binário para hexadecimal

usando o número binário 101101111 como exemplo:

- separar em grupos de 4 bits(da direita p/ esquerda)

  1011 01111 1

- adicione zeros a esquerda do último grupo p/ completá-lo

  00001 0110 1111

- converta cada grupo

  0001 = 1
  0110 = 6
  1111 = F

- combine os dígitos hexadecimais: 16F

  Portanto, 101101111 em binário e 16F em hexadecimal

2. Números com Sinal e Sem Sinal

Os números binários podem ser representados em formato de complemento de 1 ou complemento de 2 para lidar com números negativos. No complemento de 1, o número é invertido, e no complemento de 2, o número invertido tem 1 agregado ao final. O complemento de 2 é amplamente utilizado porque simplifica as operações aritméticas no hardware.

3.1. complemento de 1

O **complemento de um** de um número binário é obtido simplesmente invertendo todos os bits do número, ou seja, trocando todos os 0s por 1s e todos os 1s por 0s.

Como funciona:

1. Escreva o número binário
2. Inverta todos os bits(0s se tornam 1s e 1s se tornam 0s)

exemplo:

vamos converter o número decimal 5 para seu complemento de um, usando 4bits.

1. O número 5 em binário(4 bits) é 0101
2. O complemento de um de 0101 é 1010

Se o número original fosse negativo, a interpretação de 1010 em complemento de um seria -5.

3.2. complemento de 2

O **complemento de dois** é uma melhoria em relação ao complemento de um e é a forma mais comum de representar números negativos em binário. Ele é obtido invertendo todos os bits do número (como no complemento de um) e depois somando 1 ao resultado.

Como Funciona:

1. Escreva o número binário.
2. Inverta todos os bits (como no complemento de um).
3. Adicione 1 ao resultado.

Exemplo

Vamos converter o número decimal 5 para seu complemento de dois, usando 4 bits.

1. O número 5 em binário (4 bits) é **0101**.
2. O complemento de um de 0101 é **1010**.
3. Adicionando 1 a 1010:

$$1010 + 0001 = 1011$$

Portanto, o complemento de dois de 0101 é **1011**.

4. Sinal e Magnitude

Outra forma de representar números negativos é o formato de sinal e magnitude, onde o bit mais significativo indica o sinal (0 para positivo e 1 para negativo), e os outros bits representam a magnitude. No entanto, esse formato tem desvantagens, como a

necessidade de tratamento especial para o bit de sinal, levando ao seu abandono em favor do complemento de 2.

Como funciona o Sinal e Magnitude?

1. **Bit de sinal**: O bit mais significativo(MSB, o bit à esquerda) é reservado para indicar o sinal do número.

    - **0** indica que o número é positivo.
    - **1** indica que o número é negativo

2. **Magnitude**: Os bits restantes (menos significativos) representam a magnitude (o valor absoluto) do número.

Exemplo

Suponha que estamos usando 4 bits para representar números:

- **+5** em sinal e magnitude:
    - O bit de sinal é `0` (positivo).
    - A magnitude de 5 é `0101`.
    - Portanto, +5 é representado como **0101**.
- **-5** em sinal e magnitude:
    - O bit de sinal é `1` (negativo).
    - A magnitude de 5 é `0101`.
    - Portanto, -5 é representado como **1101**.

5. Cálculo de Deslocamento na Instrução J

Na arquitetura MIPS, a instrução de salto (Jump - J) utiliza um campo de 26 bits para o endereço de destino, que é deslocado para a esquerda em 2 bits (ou seja, multiplicado por 4). Isso significa que o endereço efetivo é obtido ao concatenar os 26 bits com os dois bits mais à direita como zero.

5.1. Estrutura da Instrução J

A instrução J na MIPS possui a seguinte estrutura:

- **Opcode (6 bits)**: Especifica que a instrução é um salto (J).
- **Endereço de Destino (26 bits)**: Um campo que especifica a localização para onde o controle do programa deve saltar.

5.2. Cálculo do Endereço de Destino

O endereço de destino real no qual o programa vai saltar é calculado utilizando os 26 bits do campo de endereço na instrução de salto. Como o MIPS utiliza endereços **word-aligned** (endereços múltiplos de 4 bytes), o cálculo do endereço de salto é realizado da seguinte forma:

Deslocamento dos 26 bits para a esquerda:

- O campo de 26 bits na instrução J é deslocado 2 bits à esquerda. Isso é equivalente a multiplicar o valor por 4, garantindo que o endereço resultante seja word-aligned (múltiplo de 4).

Concatenar com os 4 bits mais significativos do PC:

- O endereço de destino completo tem 32 bits. Para formar o endereço completo, os 26 bits deslocados são concatenados (colocados lado a lado) com os 4 bits mais significativos do Program Counter (PC). Isso mantém o salto dentro do mesmo segmento de 256 MB onde a instrução J foi executada.
- A estrutura do PC (Program Counter) é dividida assim:
    - **PC[31:28]**: Os 4 bits mais significativos do PC.
    - **Offset[25:0]**: Os 26 bits fornecidos pela instrução J, deslocados para a esquerda.

5.3. Fórmula para o Cálculo

$$Endereço\ de\ salto\ =\ PC[31:28]||(Offset\ <<\ 2)$$

PC[31:28]: São os 4 bits mais significativos do PC.
Offset[25:0]: São os 26 bits de endereço deslocados 2 bits à esquerda (ou seja, multiplicados por 4).
`||` significa "concatenar", ou seja, colocar um conjunto de bits ao lado de outro.

Exemplo Prático

Vamos supor que temos uma instrução J com os seguintes valores:

- **PC (Program Counter)**: 0x00400000
- **Campo de endereço de 26 bits**: 0x3F0000

Passos para calcular o endereço de salto:

1. **Deslocar os 26 bits para a esquerda em 2 posições**:

$$0x3F0000\ \times\ 4\ =\ 0xFC0000$$

2. **Concatenar com os 4 bits mais significativos do PC**:
    - Os 4 bits mais significativos de 0x00400000 são 0000.
    - Concatenando: 0000 || FC0000 = **0x00FC0000**

Portanto, o endereço efetivo de salto é **0x00FC0000**.

Resumo

A instrução J na MIPS usa um campo de 26 bits para o endereço de destino.

Esse campo é deslocado 2 bits à esquerda (multiplicado por 4) para garantir que o salto seja word-aligned.

O endereço final de 32 bits é obtido concatenando os 26 bits deslocados com os 4 bits mais significativos do PC atual.

6. Branch (Desvio)

As instruções de desvio (branch) no MIPS permitem que o fluxo de execução mude condicionalmente. O endereço de desvio é calculado somando o valor do campo de deslocamento (sign-extended) ao endereço da próxima instrução.

6.1. Estrutura da Instrução de Desvio (Branch)

As instruções de desvio em MIPS, como `beq` (branch if equal) e `bne` (branch if not equal), têm o seguinte formato:

- **Opcode (6 bits)**: Especifica que a instrução é um desvio (ex: `beq` ou `bne`).
- **rs (5 bits)**: Registrador fonte 1.
- **rt (5 bits)**: Registrador fonte 2.
- **Offset (16 bits)**: Campo de deslocamento que indica o quanto o programa deve saltar se a condição for verdadeira.

6.2. Funcionamento do Desvio Condicional

Comparação dos Registradores:

- A instrução de desvio compara os valores armazenados nos registradores `rs` e `rt`.
- Se a condição especificada for verdadeira (por exemplo, se `rs == rt` no caso de `beq`), o desvio ocorrerá.

Cálculo do Endereço de Desvio:

- **Offset**: A instrução contém um campo de deslocamento de 16 bits, que é usado para calcular o endereço de desvio.

- **Sign-Extension**: Como o campo de deslocamento tem 16 bits, ele é primeiro **sign-extended** para 32 bits. Isso significa que o bit mais significativo (bit de sinal) é estendido para os 16 bits adicionais, preservando o valor negativo ou positivo do deslocamento.
- **Multiplicação por 4**: O deslocamento é multiplicado por 4 (ou deslocado 2 bits à esquerda) para que o desvio seja feito em termos de palavras (word-aligned).
- **Cálculo do Endereço de Desvio**: O endereço de desvio é calculado somando o valor do campo de deslocamento (já sign-extended e multiplicado por 4) ao endereço da próxima instrução (que já está no PC + 4).

$$Endereço\ de\ Desvio\ = PC + 4 + (Offset \times 4)$$

Execução do Desvio:

- Se a condição da instrução for verdadeira, o valor calculado é carregado no Program Counter (PC), redirecionando o fluxo de execução para o novo endereço.
- Se a condição for falsa, o PC simplesmente avança para a próxima instrução sequencial, como de costume.

Exemplo Prático

Vamos considerar um exemplo com a instrução `beq`:

- Instrução: `beq $t0, $t1, offset`
- Suponha que o valor no `PC` (Program Counter) atual seja `0x00400000`, e o `offset` fornecido seja `0x0004` (16 bits).

Passos para Calcular o Endereço de Desvio:

Verificar Condição: Primeiro, o MIPS verifica se os valores nos registradores `$t0` e `$t1` são iguais.

Sign-Extension:

- O offset `0x0004` é sign-extended para 32 bits, resultando no mesmo valor `0x00000004` (porque o valor original é positivo).

Multiplicação por 4:

- O offset é multiplicado por 4:

$$0x00000004 \times 4 = 0x00000010$$

Calcular o Endereço de Desvio:

- O endereço da próxima instrução é `PC + 4 = 0x00400004`.
- O endereço de desvio final é:

$$0x00400004 + 0x00000010 = 0x00400014$$

Desvio:

- Se $t0 e $t1 forem iguais, o PC será atualizado para 0x00400014, e a próxima instrução a ser executada estará nesse endereço.

- Se $t0 e $t1 não forem iguais, o PC continuará com o valor 0x00400004 (o próximo endereço sequencial).

Resumo

- **Instruções de desvio** permitem alterar o fluxo de execução condicionalmente.
- O **cálculo do endereço de desvio** envolve sign-extension do offset de 16 bits, seguido de uma multiplicação por 4 e adição ao endereço da próxima instrução (PC + 4).
- **Desvios** são essenciais para a implementação de estruturas de controle em programas, como loops e condicionais.

7. Convertendo uma instrução MIPS para binário e hexadecimal

Converter uma instrução MIPS para binário e de binário para hexadecimal envolve vários passos. Vamos abordar cada um deles com exemplos.

7.1. Estrutura Geral de uma Instrução MIPS

As instruções MIPS são geralmente de três tipos: **R-type**, **I-type**, e **J-type**. Cada uma tem uma estrutura específica:

R-type (Register Type):

- Usada para operações aritméticas e lógicas.
- Formato: `opcode (6 bits)`, `rs (5 bits)`, `rt (5 bits)`, `rd (5 bits)`, `shamt (5 bits)`, `funct (6 bits)`.

I-type (Immediate Type):

- Usada para operações com valores imediatos, acessos à memória e desvios condicionais.
- Formato: `opcode (6 bits)`, `rs (5 bits)`, `rt (5 bits)`, `immediate (16 bits)`.

J-type (Jump Type):
- Usada para instruções de salto.
- Formato: opcode (6 bits), address (26 bits).

7.2. Passos para Converter uma Instrução MIPS para Binário

Exemplo 1: Instrução R-Type

Vamos converter a instrução **`add $t1, $t2, $t3`** para binário e hexadecimal.

- **Opcode**: 6 bits, para `add`, é `000000`.
- **rs**: Registrador origem 1, `$t2` = registrador 10, binário `01010`.
- **rt**: Registrador origem 2, `$t3` = registrador 11, binário `01011`.
- **rd**: Registrador destino, `$t1` = registrador 9, binário `01001`.
- **shamt**: Shift amount, para `add` é `00000` (não usado).
- **funct**: 6 bits, para `add`, é `100000`.

Binário:

```
opcode | rs    | rt    | rd    | shamt | funct

000000 | 01010 | 01011 | 01001 | 00000 | 100000
```

Concatenando os campos, obtemos:

Binário: `000000 01010 01011 01001 00000 100000`

Sem espaços: 00000001010010110100100000100000

Conversão para Hexadecimal

Para converter o binário para hexadecimal, divida o binário em grupos de 4 bits e converta cada grupo:

```
0000 0001 0100 1011 0100 1000 0010 0000

  0    1    4    B    4    8    2    0
```

Hexadecimal: `0x014B4820`

Exemplo 2: Instrução I-Type

Vamos converter a instrução **`addi $t1, $t2, 10`** para binário e hexadecimal.

- **Opcode**: 6 bits, para `addi`, é `001000`.
- **rs**: Registrador origem, `$t2` = registrador 10, binário `01010`.
- **rt**: Registrador destino, `$t1` = registrador 9, binário `01001`.
- **Immediate**: Valor imediato 10, em binário é `0000000000001010`.

Binário:

```
opcode | rs    | rt    | immediate

001000 | 01010 | 01001 | 0000000000001010
```

Sem espaços: **00100001010010010000000000001010**

Conversão para Hexadecimal:

```
0010 0001 0100 1001 0000 0000 0000 1010

 2    1    4    9    0    0    0    A
```

Hexadecimal: `0x2149000A`

Exemplo 3: Instrução J-Type

Vamos converter a instrução `j 0x00400024` para binário e hexadecimal.

- **Opcode: 6 bits, para `j`, é `000010`.**
- **Address: O endereço é `0x00400024`. Em binário, 26 bits para o endereço são `00000000010000000000001000`.**

Binário:

```
opcode | address

000010 | 00000000010000000000001000
```

Sem espaços: **00001000000000010000000000001000**

Conversão para Hexadecimal:

```
0000 1000 0000 0001 0000 0000 0010 00

 0    8    0    1    0    0    2    8
```

Hexadecimal: `0x08010008`

Resumo dos Passos

1. **Identifique o formato da instrução** (R-type, I-type, ou J-type).
2. **Divida a instrução nos campos** (opcode, registradores, shamt, funct, immediate, address).
3. **Converta cada campo para binário**.
4. **Junte os campos binários** para formar a instrução completa.
5. **Converta o binário em hexadecimal** para uma representação mais compacta.

8. Convertendo uma instrução binário e hexadecimal para MIPS

Convertendo com um exemplo:

1. Primeira Instrução

Hexadecimal: `0000 0001 0100 1011 0100 1000 0010 0000`

Passo 1: Converter para Binário

- **Binário:** `000000 01010 01011 01001 00000 100000`

Passo 2: Interpretar os Campos

- **Opcode (6 bits):** `000000` (Tipo R)
- **rs (5 bits):** `01010` (Registrador `$t2`)
- **rt (5 bits):** `01011` (Registrador `$t3`)
- **rd (5 bits):** `01001` (Registrador `$t1`)
- **shamt (5 bits):** `00000` (Shift amount, não utilizado aqui)
- **funct (6 bits):** `100000` (Função `add`)

Instrução em Assembly

- **Instrução:** `add $t1, $t2, $t3`

2. Segunda Instrução

Hexadecimal: `0010 1000 0001 000010000 0000 0000 1111`

Passo 1: Converter para Binário

- **Binário:** `001010 00001 00001 00000 00000 00001111`

Passo 2: Interpretar os Campos

- **Opcode (6 bits):** `001010` (Tipo I, operação `slti`)
- **rs (5 bits):** `00001` (Registrador `$at`)
- **rt (5 bits):** `00001` (Registrador `$at`)
- **Immediate (16 bits):** `0000 0000 0000 1111` (Valor imediato `0x000F` ou `15` em decimal)

Instrução em Assembly

- **Instrução:** `slti $at, $at, 15`

3. Terceira Instrução

Hexadecimal: `0001 0110 0111 10011111 1111 1111 1110`

Passo 1: Converter para Binário

- **Binário:** `000101 00111 10011 1111 1111 1111 1110`

Passo 2: Interpretar os Campos

- **Opcode (6 bits):** `000101` **(Tipo I, operação** `bne`**)**
- **rs (5 bits):** `00111` **(Registrador** `$a3`**)**
- **rt (5 bits):** `10011` **(Registrador** `$s3`**)**
- **Offset (16 bits):** `1111 1111 1111 1110` **(Deslocamento imediato** `-2`**)**

Cálculo do Endereço de Desvio

O endereço de desvio é calculado como:

$$Endereço\ de\ Desvio\ =\ PC\ +\ 4\ +\ (Offset\ \times\ 4)$$

Considerando que o PC é 0x00000008 (o endereço da instrução seguinte), e o offset é `-2`:

$$Endereço\ de\ Desvio\ =\ 0x00000008\ +\ 4\ +\ (-\ 2{\times}4)\ =\ 0x00000008\ +\ 4\ -\ 8\ =\ 0x00000004$$

Esse endereço corresponde ao label `L2`.

Instrução em Assembly

- **Instrução:** `bne $a3, $s3, L2`

4. Quarta Instrução

Hexadecimal: `0000 1000 0000 0000 0000 0000 0000 0010`

Passo 1: Converter para Binário

- **Binário:** `000010 00000 00000 00000 00000 00000010`

Passo 2: Interpretar os Campos

- **Opcode (6 bits):** `000010` (Tipo J, operação `j`)
- **Address (26 bits):** `00000 00000 00000 00000 00000010` (Valor do endereço `0x00000008`)

Instrução em Assembly

- **Instrução:** `j 0x00000008` (que corresponde ao label `L3`)

Resumo das Instruções em Assembly

- **Endereço 0x00000000:** `add $t1, $t2, $t3`
- **Endereço 0x00000004:** `slti $at, $at, 15`
- **Endereço 0x00000008:** `bne $a3, $s3, L2`
- **Endereço 0x0000000C:** `j 0x00000008` **(label** `L3`**)**

9. Suporte a Procedimentos no Hardware do Computador

O MIPS e outras arquiteturas de processadores oferecem suporte a chamadas de procedimento através de convenções específicas que utilizam registradores para passar argumentos e armazenar endereços de retorno. Isso permite que o hardware gerencie eficientemente o fluxo de controle durante a execução de funções .

8.1. Suporte a Procedimentos no MIPS

1. **Registradores Dedicados para Procedimentos**
   - **Registradores de Argumento** (`$a0` - `$a3`): Usados para passar os primeiros quatro argumentos de uma função.
   - **Registradores de Retorno de Valores** (`$v0`, `$v1`): Usados para armazenar o valor de retorno de uma função.
   - **Registrador de Endereço de Retorno** (`$ra`): Armazena o endereço de retorno, ou seja, o endereço da instrução para a qual o controle deve voltar após a conclusão da função.
   - **Registradores Temporários** (`$t0` - `$t9`): Usados para armazenar valores temporários. Não são preservados entre chamadas de funções.
   - **Registradores Salvos** (`$s0` - `$s7`): Usados para armazenar valores que precisam ser preservados entre chamadas de funções.

2. **Chamadas de função**
   - **Passagem de Argumentos**: Os primeiros quatro argumentos de uma função são passados usando os registradores `$a0` a `$a3`. Se houver mais de quatro argumentos, os valores adicionais são passados via pilha.
   - **Salvar o Endereço de Retorno**: O registrador `$ra` (return address) é usado para armazenar o endereço da próxima instrução após a chamada da função, para que o controle possa retornar corretamente após a execução da função.
   - **Salvamento de Contexto**: Antes de uma função modificar qualquer registrador que precise ser preservado (como os registradores `$s0` - `$s7`), o conteúdo desses registradores deve ser salvo na pilha. Após a execução da função, esses valores são restaurados da pilha.

3. **Retorno da Função**
    - **Retornar o Valor**: O valor de retorno da função é armazenado nos registradores `$v0` e `$v1`.
    - **Restaurar o Contexto**: Antes de retornar ao chamador, a função restaura os registradores que foram salvos no início da execução.
    - **Retorno ao Chamador**: O registrador `$ra` contém o endereço para o qual o controle deve retornar. A instrução `jr $ra` (jump register) é usada para retornar à instrução seguinte à chamada da função.

4. **Uso da Pilha**
    - **Armazenamento Adicional**: Se uma função precisar de mais registradores do que os disponíveis ou precisar armazenar temporariamente valores entre chamadas de função, a pilha é usada.
    - **Quadro de Pilha (Stack Frame)**: Cada chamada de função cria um novo quadro na pilha que pode conter os argumentos adicionais, valores temporários, e os endereços de retorno.

Exemplo de Suporte a Procedimentos

```
add_numbers:
    add $v0, $a0, $a1   # Soma $a0 e $a1, resultado em $v0
    jr $ra              # Retorna ao chamador
```

- **Passagem de Argumentos**: Os números a serem somados são passados nos registradores `$a0` e `$a1`.
- **Execução da Função**: A função soma esses dois valores e coloca o resultado no registrador `$v0`.
- **Retorno**: A função usa a instrução `jr $ra` para retornar ao endereço armazenado em `$ra`, que é o endereço da instrução seguinte à chamada da função.

Vantagens do Suporte a Procedimentos

- **Modularidade**: Facilita a escrita de código modular, onde as funções podem ser definidas uma vez e chamadas várias vezes.
- **Eficiência**: O uso de registradores dedicados e da pilha permite que o processador manipule chamadas de função e retornos de maneira rápida e eficiente.
- **Preservação de Estado**: A convenção de uso de registradores temporários e salvos, junto com a pilha, assegura que o estado do programa antes da chamada de função seja preservado e restaurado corretamente.

Resumo

O suporte a procedimentos no MIPS é baseado em um conjunto de convenções de uso de registradores e da pilha, permitindo que funções sejam chamadas e retornem valores de maneira eficiente. Isso inclui a passagem de argumentos, o armazenamento de endereços de retorno, e a preservação de contextos através do uso adequado da pilha e dos registradores, permitindo um gerenciamento eficiente do fluxo de controle durante a execução do código.

10. Desempenho

O desempenho de um programa depende de vários fatores, incluindo o algoritmo utilizado, a linguagem de programação, o compilador, o processador, o sistema de memória, e o sistema de entrada/saída. Esses fatores determinam o número de instruções que o processador executa e a velocidade dessas execuções.

$$Tempo\ de\ ciclo\ computador\ A\ = \frac{Ciclos\ de\ clock\ da\ CPU\ para\ um\ programa}{Taxa\ de\ clock\ de\ A}$$

$$Tempo\ de\ ciclo\ computador\ B\ = \frac{Ciclos\ de\ clock\ da\ CPU\ para\ um\ programa}{Taxa\ de\ clock\ de\ B}$$

$$CPI\ Média\ de\ A\ = \frac{Ciclos\ de\ clock\ de\ CPU\ de\ A}{Contagem\ de\ instruções\ de\ A}$$

$$CPI\ Média\ de\ B\ = \frac{Ciclos\ de\ clock\ de\ CPU\ de\ B}{Contagem\ de\ instruções\ de\ B}$$

$$Tempo\ de\ execução\ do\ computador\ A\ = I \times CPI\ Média\ de\ A \times Tempo\ de\ ciclo\ computador\ A$$

$$Tempo\ de\ execução\ do\ computador\ B\ = I \times CPI\ Média\ de\ B \times Tempo\ de\ ciclo\ computador\ B$$

$$Desempenho\ relativo\ = \frac{Tempo\ de\ execução\ do\ computador\ A}{Tempo\ de\ execução\ do\ computador\ B}$$

Tabela de Conversão Binário para Hexadecimal

0000 = 0

0001 = 1

0010 = 2

0011 = 3

0100 = 4

0101 = 5

0110 = 6

0111 = 7

1000 = 8

1001 = 9

1010 = A

1011 = B

1100 = C

1101 = D

1110 = E

1111 = F

Referências

GPT, CHAT - https://chatgpt.com/share/a3b38ac2-2d1a-4abc-9ba2-7e8c8c0e2afb

P1 do Gustavo 2023
P1 - Google Drive

David A. Patterson , John L. Hennessy - ORGANIZAÇÃO E PROJETO DE COMPUTADORES
livroAC.pdf

Tabela MIPS do Delair
Guia MIPS V AC.pdf